

**Name: Oluleye Oluwadunsin Stephen**

**Matric No: 220404014**

**Csc426 weekly dev- Weekend 3**

## **SOFTWARE REQUIREMENTS SPECIFICATION (SRS)**

### **ISSUES REPORTING OUTLET (IRO)**

#### **Chapter One**

##### **1.1 Purpose**

The Issues Reporting Outlet (IRO) is a web-based application developed to provide an efficient platform for reporting community-related issues within Adekunle Ajasin University (AAUA), Akungba-Akoko, and surrounding communities such as Iwaro, Etioro, Ayegunle, and neighboring areas. The system enables students, residents, and visitors to report issues affecting their environment through an online platform.

The primary purpose of the system is to simplify the process of reporting community issues while maintaining a centralized database where submitted reports can be stored and managed. The application also serves as a medium for improving communication between members of the community and the responsible authorities by providing an organized approach to issue reporting.

##### **1.2 Scope**

The Issues Reporting Outlet (IRO) is designed as a simple web application that focuses on the reporting and management of community issues. The scope of the system includes the following:

- Allow users to report community issues through an online form.
- Store reported issues securely in a SQLite database.
- Display previously submitted issues for monitoring purposes.
- Provide a simple and user-friendly interface that is easy to navigate.
- Serve as a centralized platform for documenting community-related issues.

The current version of the system focuses on basic issue reporting and does not include advanced features such as user authentication, email notifications, or file uploads.

##### **1.3 Objectives**

The major objectives of the Issues Reporting Outlet (IRO) are:

- To simplify the process of reporting community issues.
- To improve communication between community members and relevant authorities.

- To maintain accurate records of reported issues.
- To encourage community participation in identifying and reporting environmental problems.
- To replace manual reporting methods with a simple digital reporting system.

## **Chapter Two**

### **2.0 Overall Description**

The Issues Reporting Outlet (IRO) is a standalone web application that operates independently through a web browser. The system is developed using modern web technologies and follows a client-server architecture, where users interact with the application through a browser while the backend processes requests and stores data in a SQLite database.

The technologies used for the development of the system include:

- HTML5
- CSS3
- JavaScript
- Python (Flask Framework)
- SQLite Database

### **2.2 Product Functions**

The system provides the following major functions:

- Home Page
- Report Issue Page
- Store Issue Reports
- View Reported Issues

These functions enable users to submit issue reports, store information in the database, and access previously submitted reports.

### **2.3 Users**

The Issues Reporting Outlet consists of two categories of users.

Community User

The community user can:

- Access the homepage.

- Submit issue reports.
- View previously reported issues.

#### Administrator

The administrator is responsible for monitoring the operation of the application and managing the stored issue records within the database.

## 2.4 Operating Environment

The application can operate on the following operating systems:

- Windows
- Linux
- macOS

The application is compatible with modern web browsers including:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Since the application is web-based, no additional software installation is required apart from a supported web browser.

## Chapter Three

### 3.0 Functional Requirements

The functional requirements describe the major operations that the system performs.

FR1

The system shall allow users to access the Home Page.

FR2

The system shall allow users to submit issue reports.

FR3

The system shall validate all required input fields before submission.

FR4

The system shall store submitted reports in the SQLite database.

FR5

The system shall display previously submitted issue reports.

### **3.1 Non-Functional Requirements**

#### Performance

- Pages should load quickly.
- Submitted reports should be stored within a few seconds.
- The application should provide smooth navigation between pages.

#### Security

- Required input fields shall be validated before submission.
- SQL Injection attacks shall be prevented through parameterized SQL queries.
- Invalid or incomplete data shall not be stored in the database.

#### Reliability

- Reports should remain stored after successful submission.
- Stored records should remain available unless intentionally removed.

#### Usability

- The user interface should be simple and user-friendly.
- Navigation should be clear and easy to understand.
- Users should be able to submit reports without technical knowledge.

#### Availability

The application shall remain available whenever the Flask web server is running.

### **3.2 System Components**

#### **Frontend**

##### Technologies Used

- HTML5
- CSS3
- JavaScript

Purpose: The frontend provides the graphical user interface through which users interact with the application. It is responsible for displaying pages, collecting user input, and presenting information in a simple and attractive manner.

## Backend

**Technology Used :** Python (Flask Framework)

Purpose: The backend processes user requests, validates submitted information, communicates with the SQLite database, and returns appropriate responses to the user interface.

## Database

**Technology Used:** SQLite Database

Purpose: The database stores all issue reports submitted through the application. Each report is maintained for future reference and monitoring.

### 3.3 System Interaction

The interaction between the major components of the system is illustrated below.

Community User

|



Web Browser

|



Flask Web Application

|



SQLite Database



|

Flask Web Application



|

Web Browser

The user accesses the application through a web browser. The browser communicates with the Flask application, which processes requests and stores or retrieves information from the SQLite database before returning the results to the user.

### **3.4 Assumptions**

The successful operation of the system is based on the following assumptions:

- Users have access to the Internet.
- Users have access to a modern web browser.
- The Flask server is operational.
- Users provide accurate information when submitting issue reports.

### **3.5 Limitations**

The current version of the Issues Reporting Outlet (IRO) has the following limitations:

- The system does not support file or image uploads.
- Email notifications are not implemented.
- User authentication and account management are not included.
- The application is intended for small-scale deployment.
- SQLite is suitable only for lightweight applications with a limited number of concurrent users.

### **3.6 Conclusion**

The Issues Reporting Outlet (IRO) provides a simple, efficient, and user-friendly platform for reporting community-related issues. The application improves the traditional method of reporting by allowing users to submit issues electronically while maintaining a centralized database of reports.

Developed using HTML5, CSS3, JavaScript, Python (Flask), and SQLite, the system demonstrates the practical application of web technologies in addressing real-life community challenges. Its simple architecture, ease of use, and reliable data storage make it suitable as an academic project and as a foundation for future enhancements.